

第三届中国研究生操作系统开源创新大赛

暨开放原子大赛操作系统专项赛

第 16 题：云一边一端异构计算资源调度

基于深度强化学习的 云一边一端异构计算资源管理调度方法

项目技术报告

参赛队伍：[匿名]

二〇二六年七月

目 录

摘要	1
编写依据与组织说明	2
第一章 赛题问题、数学模型与验收映射	3
1.1 问题定义	3
1.2 默认异构资源	3
1.3 评价指标	4
1.4 赛题要求到代码与证据的闭环	4
第二章 系统架构与调度物理规则	6
2.1 分层架构	6
2.2 插入式资源时间线	7
2.3 观测空间与归一化	7
2.4 Ranked 动作空间与 EFT 资源选择	7
2.5 相对 HEFT 奖励	8
第三章 技术演进：从诊断到残差调度	9
3.1 演进总览	9
3.2 策略塌缩的发现	9
3.3 归一化诊断	10
3.4 动作空间重构	10
3.5 残差式排序策略	10
3.6 Best-of-N 与精修层	11
3.7 未被采用的方向	11
第四章 最终方案与算法流程	13

4.1 完整流水线	13
4.2 正确性与不变量	13
4.3 算法定位	14
第五章 实验协议与统计方法	15
5.1 数据、训练与验证隔离	15
5.2 基线	15
5.3 重复与配对设计	15
第六章 结果与证据	16
6.1 最终方案的五次重复结果	16
6.2 计算量对齐验证	17
6.3 与 MILP 理论最优的距离	17
6.4 结构化泛化	17
6.5 大规模泛化负面结果	18
6.6 openEuler 与测试验证	19
第七章 复现、审计与工程交付	20
7.1 依赖与关键配置	20
7.2 一键运行	20
7.3 最终 LNS 方案命令	21
7.4 测试与报告审计	21
第八章 局限性、适用边界与未来工作	22
8.1 已知局限	22
8.2 后续方向	22
第九章 结论	23
参考文献	24

附录 A 关键证据索引	25
附录 B 关键代码索引	26
附录 C 匿名化与数据口径声明	27

摘要

本项目面向云、边、端异构计算资源上的有向无环图（DAG）任务调度问题。输入由带计算量与边通信量的任务图、不同算力和带宽的资源集合组成，输出为每个任务的执行顺序、资源映射与开始/结束时间，优化目标是最小化整体完成时间（makespan）。项目没有把深度强化学习作为不可解释的替代器，而是把它置于可验证的调度物理规则之上：资源时间线统一采用插入式调度，策略只负责在当前就绪任务中确定优先顺序，资源选择由最早完成时间（EFT）规则确定。

技术路线经历了固定奖励导致的策略塌缩、相对 HEFT 奖励、观测归一化、动作空间拆分、残差排序学习和随机多样化推理等阶段。最终方法以零初始化的 Residual Policy 在 HEFT upward rank 上学习有界修正量，使用 Best-of-64 构造初解，随后执行保持前驱约束的单任务重定位，并以 best-only 的破坏-修复大邻域搜索（LNS）进行精修。该精修层只接受严格降低 makespan 的候选，因此对同一初解具有“不退化”保证。

在固定的 20 个验证场景上，最终方案经 5 次系统熵种子的配对重复评测得到 $\text{mean_ratio} = 0.920890 \pm 0.001729$ ，5 轮均在 20/20 个场景反超 HEFT。与同轮原始 Residual Best-of-64 相比，配对差值为 -0.029925 ± 0.003080 ，双侧配对 t 检验 $p = 2.656851 \times 10^{-5}$ 。为排除“仅因计算量增加”的解释，又以略高推理耗时的纯 Residual Best-of-128 对齐预算；其结果为 0.942048 ± 0.002063 ，而 LNS 方案仍为 0.920890 ± 0.001729 ，配对检验 $p = 2.072081 \times 10^{-6}$ 。上述数据来自 [evaluation/results/autonomous_exploration/direction2_lns/direct_vs_residual_paired_summary.json](#) 与 [evaluation/results/autonomous_exploration/compute_matched_sampling/paired_comparison_summary.json](#)。

项目同时保留 HEFT、MILP、Hybrid 等基线，提供 Gymnasium 环境、MaskablePPO 训练、固定场景评测、统计审计、Windows/Linux/openEuler 一键运行与测试脚本。报告如实呈现大规模任务与宽并行 DAG 上的泛化不足，避免把局部有效结论外推为普遍最优。

关键词：云边端协同；异构资源调度；DAG；深度强化学习；残差调度；大邻域搜索；可复现性

编写依据与组织说明

本报告在动笔前先研究了赛事材料、公开获奖作品与机器学习报告规范，并据此确定“赛题对齐—实现机制—技术演进—统计证据—复现与边界”的组织方式。

1. 赛事官方作品提交说明要求交付源代码及 README、项目说明书、PPT 和演示材料，并允许在官方模板基础上按项目特点调整。本报告使用 docs/references/操作系统开源创新大赛项目说明书_官方模板.docx 的页面与封面规范，但不机械照搬其章节顺序。来源：中国研究生创新实践系列大赛管理平台，“第二届中国研究生操作系统开源创新大赛作品提交说明”。
2. 官方评审材料强调设计方案、实现、运行与测试结果、演示、创新性、可读性、准确性。报告因此在前半部分明确给出“赛题要求—代码实体—可验证证据”的映射，而不是只叙述算法概念。
3. 公开可核验的往届获奖项目 ServerlessOS 与“基于粒子系统的动态图形渲染”说明书普遍采用“问题背景—总体设计—关键实现—实验—使用说明”的主线；本报告借鉴其层次化架构图、关键模块表和复现命令写法，不复制其正文内容。
4. 清华大学《研究生学位论文写作指南》强调论点、数据、图表、单位、参考文献的规范表达；Pineau 等关于机器学习可复现性的研究强调公开配置、随机性、统计口径与运行证据。报告因此把单次结果与重复统计严格分开，并专设“证据索引”和“局限性”章节。

与旧版相比，第二版不再按实验发生时间简单铺陈，而是先建立数学问题、赛题接口和系统边界，再用诊断链解释技术为何演进，最后将最终结论、反例、复现步骤和原始文件一一绑定。这样既便于评委快速核查，也能避免“结果很好但不知道对应哪段代码”的断裂。

第一章 赛题问题、数学模型与验收映射

1.1 问题定义

设任务图为 DAG $G=(V,E)$ 。节点 $i \in V$ 具有计算量 w_i ，有向边 $(i,j) \in E$ 具有数据量 d_{ij} 。资源集合 R 包含云、边和端节点；资源 r 具有算力 p_r 和带宽 b_r 。调度器需要决定任务执行顺序、资源映射 $x_i \in R$ 、开始时间 s_i 与完成时间 f_i 。

任务 i 在资源 r 上的执行时长为：

```
[text]
duration(i,r) = w_i / p_r
```

若前驱 i 与后继 j 分配到不同资源，则通信时延由数据量与有效带宽决定；同资源通信时延为零。任务 j 的开始时间必须满足：

```
[text]
s_j >= max_{i in pred(j)} (f_i + communication(i,j,x_i,x_j))
```

同一资源上的执行区间不得重叠。目标函数为：

```
[text]
minimize C_max, where C_max = max_i f_i
```

代码对应 env/dag_generator.py、env/resource_config.py、env/scheduling_utils.py 与 baselines/milp_optimal_scheduler.py。MILP 基线将资源分配、开始时间与不重叠顺序同时交给求解器，而非预先固定任务顺序。

1.2 默认异构资源

默认资源配置来自 configs/resource_default.yaml。

资源	层级	compute_power	bandwidth
cloud_0	cloud	100	1000
edge_0	edge	30	200

资源	层级	compute_power	bandwidth
edge_1	edge	25	150
device_0	device	5	20

该配置故意保留显著异构性：最快云节点并不必然产生最短全局 makespan，因为资源竞争、通信代价和 DAG 并行度会改变局部最优选择。

1.3 评价指标

对场景 k ，定义：

```
[text]
ratio_k = makespan_method(k) / makespan_HEFT(k)
mean_ratio = average_k(ratio_k)
```

$ratio < 1$ 表示该场景反超 HEFT； $ratio = 1$ 表示相同； $ratio > 1$ 表示弱于 HEFT。报告同时给出反超场景数、重复评测样本标准差、配对差值与双侧配对 t 检验。评测实现见 `evaluation/evaluate.py`、`evaluation/evaluate_bestofn.py` 与 `evaluation/analyze_residual_lns_direct_repeats.py`。

1.4 赛题要求到代码与证据的闭环

赛题环节	本项目定义	关键代码	验证证据
问题建模	DAG 节点计算量、边通信量、异构资源算力/带宽	<code>env/dag_generator.py</code> 、 <code>env/resource_config.py</code>	<code>tests/test_env.py</code> 、 <code>tests/test_milp_optimal_scheduler.py</code>
环境状态	任务特征、邻接矩阵、就绪映射、资源状态、全局进度	<code>env/scheduling_env.py</code> 、 <code>env/observation_normalizer.py</code>	<code>tests/test_env.py</code> 、 <code>tests/test_gat_features_extractor.py</code>
动作空间	仅选择当前就绪任务；资源由 EFT 确定	<code>env/scheduling_env_ranked.py</code>	<code>tests/test_scheduling_env_ranked.py</code>
合法性	Action mask + 拓扑就绪条件 + 插入式资源时间线	<code>action_masks()</code> 、 <code>find_earliest_slot()</code>	<code>tests/test_scheduling_consistency.py</code>
奖励	小步惩罚 + 终局相对 HEFT 差距	<code>reward_mode=relative_heft</code>	<code>tests/test_reward_shaping.py</code>
策略	HEFT rank 加零初始化解残差 logits	<code>env/scheduling_env_residual.py</code> 、 <code>policies/residual_ranking_policy.py</code>	<code>tests/test_residual_scheduling.py</code>

赛题环节	本项目定义	关键代码	验证证据
训练	MaskablePPO , 配置驱动	training/train_ppo.py 、 training/configs/ppo_mlp_residual.yaml	训练日志与 checkpoint
评估	固定场景、HEFT 对照、重复评测、配对检验	evaluation/	evaluation/results/
理论参照	小规模 MILP 精确/限时求解	baselines/milp_optimal_scheduler.py	evaluation/results/milp_optimal_comparison.json
跨平台	Windows 、 Linux/openEuler	一键脚本与 Docker 验证	evaluation/results/open Euler_validation/

第二章 系统架构与调度物理规则

2.1 分层架构

统一调度接口贯穿训练、评测与证据链

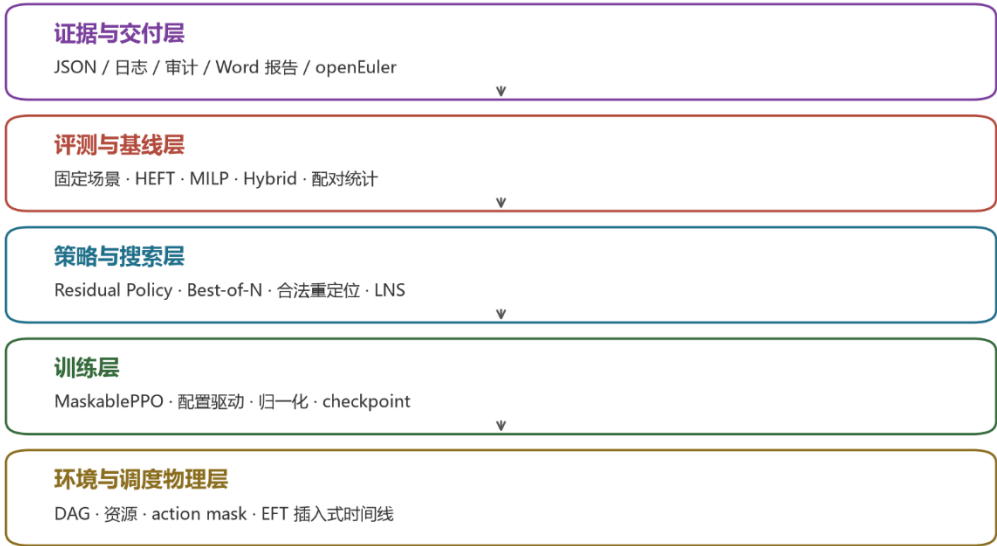


图 1 系统分层架构

项目按职责分为环境层、基线层、策略层、训练层、评测层和证据层。所有调度器继承 scheduler_interface.py 中的统一接口，返回 task_id -> (resource_id, start_time, finish_time)，从而使 HEFT、MILP、Hybrid、RL 和 LNS 在同一 makespan 计算函数下比较。

目录	主要职责	代表文件
env/	DAG、资源、观测、奖励、动作掩码与插入调度	env/scheduling_env.py env/scheduling_env_ranked.py env/scheduling_utils.py
baselines/	HEFT、MILP、Hybrid	baselines/heft_scheduler.py baselines/milp_optimal_scheduler.py
policies/	MLP/GAT、Residual、Best-of-N 与搜索精修	policies/residual_ranking_policy.py policies/residual_local_search_scheduler.py
training/	配置驱动训练、BC、课程学习	training/train_ppo.py training/behavior_cloning.py
evaluation/	固定场景、泛化、精确解和统计分析	evaluation/evaluate.py evaluation/analyze_residual_lns_direct_repeats.py
diagnostics/	数据、梯度、mask、归一化与结果审计	diagnostics/bc_dataset_sanity_check.py diagnostics/technical_report_result_audit.py
tests/	物理一致性、策略接口与回归测试	tests/test_env.py tests/test_scheduling_consistency.py

2.2 插入式资源时间线

初版环境只把新任务追加到 `resource.available_time` 之后，而 HEFT 会搜索资源时间线中的空隙，造成同一分配方案得到不同 `makespan`。项目把这一物理规则抽取为 `env/scheduling_utils.py::find_earliest_slot`，环境和 HEFT 共用 `ScheduledEvent(task_id,start_time,finish_time)` 列表。新任务先计算依赖数据就绪时间，再在已排序事件间寻找第一个足够大的空隙。

一致性测试 `tests/test_scheduling_consistency.py` 将 HEFT 的任务—资源分配按其内部顺序通过环境逐步重放，要求两者 `makespan` 在浮点容差内一致。该测试不是模型效果测试，而是保证比较公平性的基础测试。

2.3 观测空间与归一化

观测为 Gymnasium Dict:

字段	形状	语义	是否归一化
<code>task_features</code>	<code>[max_tasks,5]</code>	有效标志、归一化任务编号、计算量、出度、后继计算量和	连续字段是
<code>task_valid_mask</code>	<code>[max_tasks]</code>	<code>padding</code> 有效位	否
<code>task_adjacency</code>	<code>[max_tasks,max_tasks]</code>	带自环的 0/1 图结构	否
<code>ready_task_node_ids</code>	<code>[max_ready_tasks]</code>	就绪槽位到真实节点编号	否
<code>resource_features</code>	<code>[num_resources,4]</code>	可用时间、层级、算力、带宽	连续字段是
<code>global_features</code>	<code>[2]</code>	完成比例、当前 <code>makespan</code>	是
<code>ready_task_upward_ranks</code>	<code>[max_ready_tasks]</code>	Residual Policy 的 HEFT 基准 logits	按 rank 统计量标准化

`env/observation_normalizer.py` 仅处理连续特征，绝不改变 `mask`、索引和邻接矩阵的离散语义。该边界由 `env/normalization_stats.json` 固化。

2.4 Ranked 动作空间与 EFT 资源选择

Joint 环境每步同时选择“就绪任务槽位 × 资源”，动作规模随就绪任务数和资源数相乘。Ranked 环境将动作简化为 `Discrete(max_ready_tasks)`：策略只选“下一个

处理哪个就绪任务”。环境随后遍历全部资源，使用相同的依赖就绪时间、执行时长与 `find_earliest_slot` 计算 EFT，选择完成时间最早的资源。

这一拆分保留了学习模型擅长的全局任务排序，把局部资源放置交给确定性规则。`tests/test_scheduling_env_ranked.py` 验证：当动作严格复现 HEFT upward-rank 顺序时，最终 makespan 与 HEFT 完全一致。

2.5 相对 HEFT 奖励

每个 episode 在独立资源配置副本上计算 HEFT 参考 makespan，避免污染 RL 环境。奖励为：

```
[text]
non-terminal: reward = -0.01
terminal:     reward = -0.01 - (current_makespan / HEFT_makespan - 1)
```

该设计把不同任务规模的终局信号统一到相对比值尺度；`tests/test_reward_shaping.py` 通过重放 HEFT 调度验证终局奖励接近 -0.01。默认 `reward_mode=raw` 保留旧行为，保证历史实验可复现。

第三章 技术演进：从诊断到残差调度

3.1 演进总览

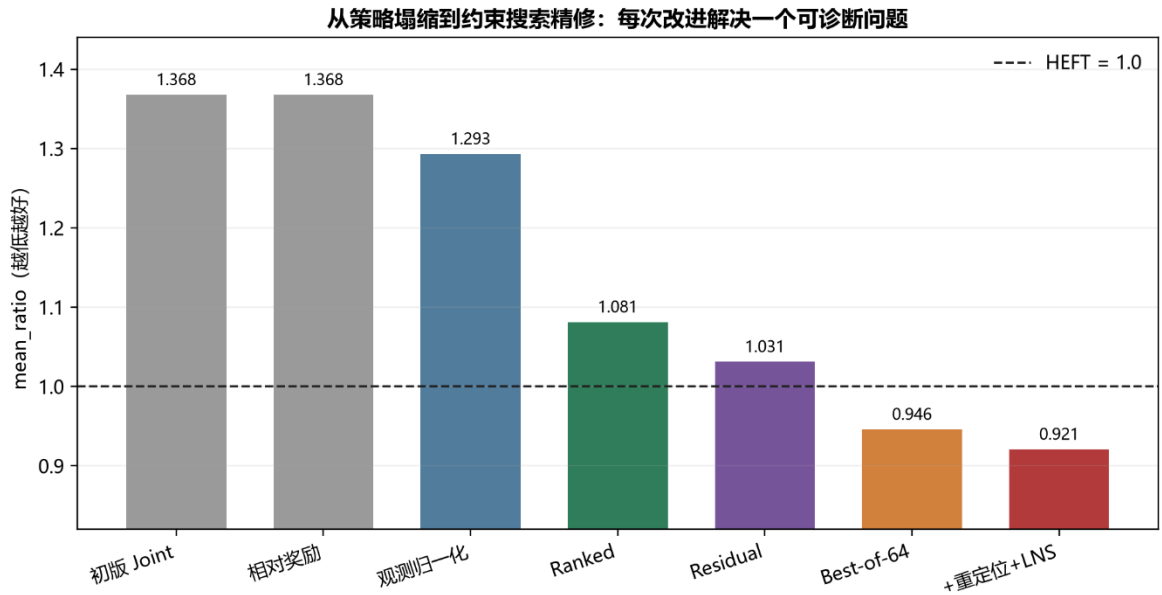


图 2 技术演进与 mean_ratio

阶段	核心改变	mean_ratio	解决/暴露的问题	数据来源
初版 Joint MLP	固定步惩罚 + 终局绝对 makespan	1.3677 62	策略塌缩，信号稀疏	evaluation/results/summary.json
Relative HEFT	终局使用相对差距	1.3677 62	单改奖励仍不足	evaluation/results/summary_reward_shaped.json
观测归一化	连续字段标准化	1.2931 90	缓解特征量纲冲突	evaluation/results/summary_mlp_normalized.json
Ranked 动作	只学任务顺序，资源用 EFT	1.0812 77	联合动作难度显著下降	evaluation/results/summary_mlp_ranked.json
Residual	HEFT rank + 有界 delta	1.0311 97	训练起点锚定 HEFT	evaluation/results/summary_mlp_residual.json
Residual Best-of-64	多次随机构造择优	0.9459 88	单次策略方差转化为候选多样性	evaluation/results/summary_mlp_residual_bestof64.json
Residual + 搜索精修	合法重定位 + best-only LNS	0.9208 ± 0.001729 90	修复高质量序列的局部排序错误	evaluation/results/autonomous_exploration/direction2_lns/direct_vs_residual_paired_summary.json

3.2 策略塌缩的发现

初版奖励为每步固定惩罚，终局再减绝对 makespan。任务规模变化使终局量纲大幅波动，而每步常数又不能区分动作质量。动作分布诊断显示，10 个诊断 DAG 的

180 个任务全部被分配到 `cloud_0`，证据见 `artifacts/diagnostic_logs/diagnostics_action_distribution_output.log`。这说明模型学到的是“选最快单节点”的短视规则，而非资源并行与通信权衡。

相对 HEFT 奖励修正了尺度，却没有自动解决观测与动作表达问题：对应正式结果仍为 1.367762，见 `evaluation/results/summary_reward_shaped.json`。这一步的价值是排除了“只改奖励即可”的假设。

3.3 归一化诊断

批量 `mask` 检查表明异构合法动作数不会发生跨样本污染；单样本训练也证明梯度、优化器和专家动作索引正常。真正转折来自小数据过拟合：同一批 93 个样本，不归一化的最好准确率为 27.957%，加入字段级标准化后达到 90.323%，证据见 `artifacts/diagnostic_logs/bc_batch_mask_and_normalization_check_output.log`。这说明计算量、带宽、时间与 0/1 特征的尺度差异阻碍了联合优化。

正式加入归一化后 `mean_ratio` 从 1.367762 降至 1.293190，数据分别见 `evaluation/results/summary.json` 与 `evaluation/results/summary_mlp_normalized.json`。归一化解决的是优化条件，而不是动作组合爆炸，因此仍未反超 HEFT。

3.4 动作空间重构

动作复杂度诊断发现，Joint 设计把任务排序和资源分配耦合成一个分类问题。Ranked 环境保留前者、确定化后者，使动作数从“就绪任务数×资源数”降为“就绪任务数”。结果由 1.293190 进一步降至 1.081277，见 `evaluation/results/summary_mlp_normalized.json` 与 `evaluation/results/summary_mlp_ranked.json`。这是整条技术路线中最关键的结构改进：它改变了学习问题，而不是继续堆叠网络规模。

3.5 残差式排序策略

Residual 环境把每个就绪任务的标准化 HEFT upward rank 暴露为 `ready_task_upward_ranks`。自定义策略 logits 为：

```
[text]
logit(task) = rank_scale * normalized_HEFT_rank(task)
```

```
+ delta_scale * tanh(MLP(task,state))
```

`policies/residual_ranking_policy.py` 将 `delta` 输出层的权重和偏置全部初始化为零。训练开始时 `delta=0`，策略排序等价于 HEFT；后续 PPO 只学习对 HEFT 的有界修正。测试 `tests/test_residual_scheduling.py` 验证零初始化、logits 加法和合法 mask。

Residual deterministic 的 `mean_ratio` 为 1.031197，优于 Ranked 的 1.081277；对应文件为 `evaluation/results/summary_mlp_residual.json` 与 `evaluation/results/summary_mlp_ranked.json`。它仍不是“保证优于 HEFT”的算法，但提供了更强、可解释的搜索起点。

3.6 Best-of-N 与精修层

`policies/rl_scheduler.py` 在 `num_samples>1` 时对同一 DAG 独立随机构造多个完整调度，保留 makespan 最小者，并可加入 deterministic 候选。Best-of-64 的固定种子单次结果为 0.945988，见 `evaluation/results/summary_mlp_residual_bestof64.json`。

最终精修层位于 `policies/residual_local_search_scheduler.py`:

1. 从 Best-of-64 得到最优合法拓扑序。
2. 枚举保持所有前驱—后继关系的单任务重定位，执行 `best-improvement`，最多按配置扫描若干轮。
3. LNS 每次破坏 2 至 4 个任务的位置锚点，用 Kahn 算法重建合法拓扑序，再做一轮局部搜索。
4. 仅当 `candidate_makespan < best_makespan - 1e-12` 时接受；相等或变差候选全部拒绝。

因此“精修后不差于同一初解”是算法设计保证。统计检验验证的是改进幅度在随机重复中是否稳定，而不是证明这个单调性质本身。报告将其定位为“在 Residual 构造器上的 best-only 搜索增强”，不把它包装成与 Residual 平行的独立路线。

3.7 未被采用的方向

负面结果同样决定最终方案边界。

方向	mean_ratio	结论	数据来源
GA + Ranked	1.0876 71	未优于 MLP Ranked	evaluation/results/summary_gat_ranked.json
Ranked + BC 热启动	1.1183 87	高 BC 准确率未转化为 PPO 最终优势	evaluation/results/summary_mlp_ranked_bc.json
Ranked + upward rank 输入特征	1.1114 98	“作为输入”不等于“作为强制加性基准”	evaluation/results/summary_mlp_ranked_with_rank.json
Ranked + 课程学习	1.1314 71	当前课程区间未改善最终分布	evaluation/results/summary_mlp_ranked_curriculum.json
Gumbel 束 + 相同后处理	0.9209 68 ± 0.001649	相对 LNS 配对 p=0.904282, 未能证明显著改善	evaluation/results/autonomous_exploration/direction3_gumbel_beam/paired_repeats_summary.json

这些结果说明：图编码、模仿学习和课程学习并非无效技术，但在本项目的数据规模、状态表达和训练预算下没有形成可重复优势，因而不进入正式方案。

第四章 最终方案与算法流程

4.1 完整流水线

最终方案：学习构造器 + 约束搜索精修器



只接受严格改善候选：同一初解上 makespan 单调不增

图 3 最终调度流水线

最终方案由“学习构造器”和“约束搜索精修器”串联：

1. 加载 training/checkpoints/ppo_mlp_residual。
2. 对同一 DAG 执行 64 次 stochastic Residual rollout，并加入 deterministic fallback。
3. 每个 rollout 只选择就绪任务，环境用 EFT 插入规则分配资源。
4. 取最小 makespan 的合法序列作为初解。
5. 执行合法单任务重定位至局部最优或达到轮次上限。
6. 执行 64 次破坏—修复 LNS；每个候选均重新执行同一 EFT 资源放置。
7. 仅保留严格改善候选，输出最终任务顺序、资源、开始与结束时间。

4.2 正确性与不变量

不变量	保障机制	测试/检查
只调度就绪任务	环境 ready list 与 action mask	tests/test_scheduling_env_ranked.py
输出保持拓扑合法	重定位前检查边两端位置；修复使用 Kahn 算法	tests/test_residual_local_search_scheduler.py
同资源任务不重叠	find_earliest_slot 插入式时间线	tests/test_scheduling_consistency.py
环境与 HEFT 物理规则一致	共用 ScheduledEvent 和插入函数	tests/test_scheduling_consistency.py
Residual 初始策略锚定 HEFT	delta 输出层零初始化	tests/test_residual_scheduling.py

不变量	保障机制	测试/检查
搜索不劣化输入解	best-only 接受 + 运行时断言	tests/test_residual_local_search_scheduler.py

4.3 算法定位

该方法不是用神经网络直接预测连续开始时间，也不是用 MILP 求解全部规模。它将三种能力组合：HEFT 提供结构化先验，Residual Policy 学习实例相关排序偏差，LNS 在合法解空间内做计算预算可控的单调精修。资源放置始终由 EFT 确定，因此模型和搜索都不会绕过操作系统调度约束。

第五章 实验协议与统计方法

5.1 数据、训练与验证隔离

最终训练配置为 `training/configs/ppo_mlp_residual.yaml`: 训练 DAG 任务规模为 8 至 25, 边密度范围为 0.2 至 0.5, padding 上限为 30, 训练步数配置为 200000, 随机种子为 123。固定验证集包含 20 个场景, 任务规模为 10、15、20、25, 每组 5 个, 验证种子从 1000000 开始。配置与场景路径分别见该 YAML 和 `evaluation/scenarios/`。

报告区分三种随机性: 训练随机种子、固定复现实验采样种子、系统熵重复评测种子。最终统计结论使用系统熵种子, 不从多次运行中挑选最好结果。

5.2 基线

1. HEFT: 使用 upward rank 排序和 EFT 插入式资源选择, 见 `baselines/heft_scheduler.py`。
2. MILP: 同时优化分配与时间, CBC 限时求解; 只有状态为 optimal 的场景才用于“距理论最优”统计, 见 `baselines/milp_optimal_scheduler.py`。
3. Hybrid: 小规模优先 MILP, 失败或超阈值回退 Residual+Best-of-N, 见 `baselines/hybrid_scheduler.py`。
4. 纯采样计算量对照: Residual Best-of-128, 其平均耗时略高于最终 LNS, 用于排除计算量混淆。

5.3 重复与配对设计

最终方案与 Residual Best-of-64 使用相同的 5 个系统熵种子, 每轮的 20 个场景共享该轮种子。令 $d_i = \text{mean_ratio_new}_i - \text{mean_ratio_baseline}_i$, 报告样本均值、样本标准差和双侧配对 t 检验。显著性阈值为 0.05; 只有均值更低且 $p < 0.05$ 才记为显著改善。

计算量对齐实验先只根据耗时校准 N: N=84 耗时偏低后, 按比例选择 N=128; 达到目标耗时范围后锁定, 不根据 `mean_ratio` 再调 N。校准与正式数据保存在 `evaluation/results/autonomous_exploration/compute_matched_sampling/`。

第六章 结果与证据

6.1 最终方案的五次重复结果

重 复	Residual Best-of-64	最终 LNS	差 值 (LNS- Residual)	Residual 反超数	LNS 反 超数	总耗时/秒
1	0.949533	0.921490	-0.028043	18	20	52.563326
2	0.948910	0.919837	-0.029073	19	20	56.751589
3	0.956776	0.922358	-0.034419	17	20	56.427768
4	0.948961	0.922360	-0.026600	19	20	54.006265
5	0.949891	0.918404	-0.031487	18	20	52.259721
均 值 ± 样 本 标 准 差	0.950814 ± 0.003358	0.920890 ± 0.001729	-0.029925 ± 0.003080	18.2/20	20.0/20	54.401734

双侧配对 t 检验 $t=-21.722925$ 、 $df=4$ 、 $p=2.656851 \times 10^{-5}$ 。完整逐轮数据与统计量来自 `evaluation/results/autonomous_exploration/direction2_lns/direct_vs_residual_paired_summary.json`。

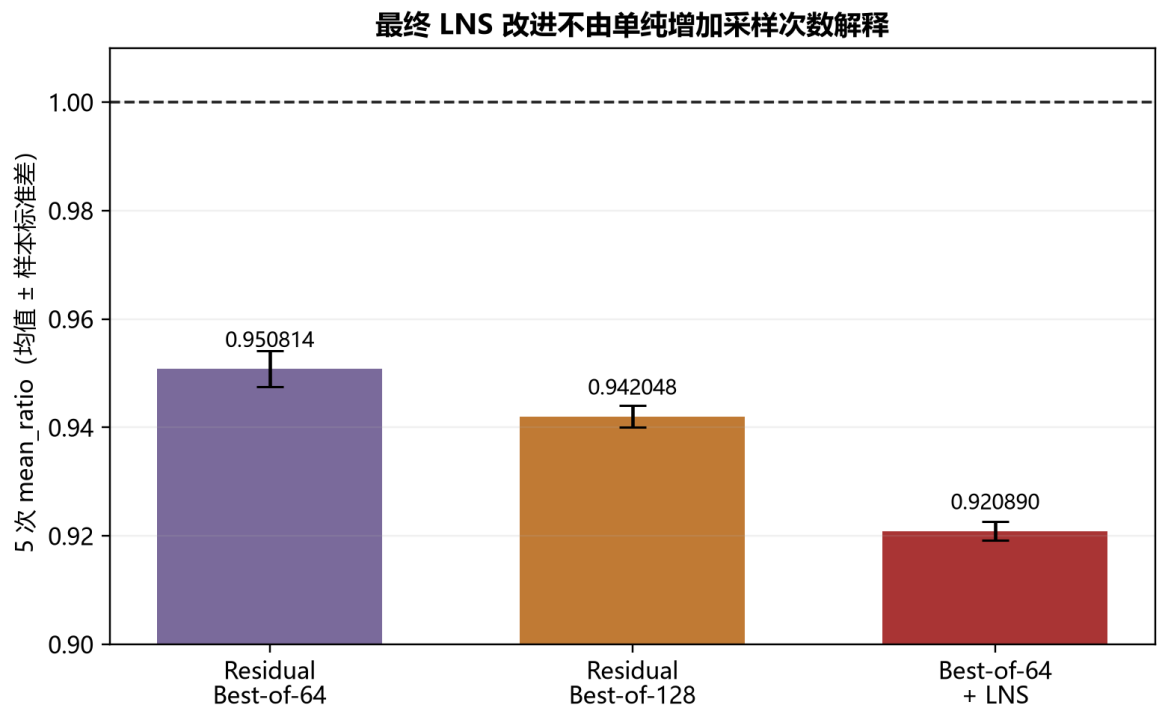


图 4 最终方案重复统计

6.2 计算量对齐验证

指标	纯 Residual Best-of-128	Residual Best-of-64 + 重定位 + LNS
mean_ratio (5 次均值 $\pm$ 样本标准差)	0.942048 ± 0.002063	0.920890 ± 0.001729
平均反超 HEFT 场景数	18.6/20	20.0/20
20 场景平均耗时/秒	56.801547	54.401734

LNS 减纯采样的配对差值为 -0.021158 ± 0.001148 ，双侧配对 t 检验 $t=-41.210742$ 、 $df=4$ 、 $p=2.072081 \times 10^{-6}$ 。纯采样耗时为 LNS 的 1.044113 倍，却仍明显更差。数据来自 `evaluation/results/autonomous_exploration/compute_matched_sampling/paired_comparison_summary.json`。因此，在当前验证集与硬件上，改进不能仅由增加 wall-clock 预算解释；LNS 的邻域结构带来了额外价值。

6.3 与 MILP 理论最优的距离

小规模 10/15 任务场景共 10 个，CBC 在限定时间内对其中 7 个证明全局最优，3 个仅得到未证明最优的可行解。仅统计 7 个已证明最优场景：

方法	方法 makespan / MILP optimum	高于最优解
HEFT	1.106807	10.6807%
Residual+Best-of-16	1.044188	4.4188%

数据来自 `evaluation/results/milp_optimal_comparison.json`。该实验早于最终 LNS，因此只证明学习构造方法在可精确求解规模上比 HEFT 更接近理论下界，不把 Best-of-16 数字冒充最终 LNS 的最优差距。3 个未证明最优场景保留 incumbent、best bound 与 gap，不纳入均值。

6.4 结构化泛化

结构化泛化使用全新种子、保持任务数在训练范围内，并改变 DAG 密度或资源异构性。该实验评测的是 Residual Best-of-64，而非后续 LNS；它用于描述学习模型本身的结构迁移能力。

场景组	mean_ratio	场景内标准差	反超 HEFT	相对原始对照差值
宽并行	1.011915	0.018469	2/5	+0.047829
深链条	0.943597	0.043357	4/5	-0.020488
同构资源	0.879572	0.071489	5/5	-0.084514
原始分布对照	0.964085	0.040774	4/5	0.000000

数据来自 `evaluation/results/structural_generalization/summary_structural_generalization.json`。宽并行组退化至略弱于 HEFT，说明当前训练分布对大量并发就绪任务覆盖不足；同构资源组明显更好，说明资源差异缩小后，任务排序优势更容易转化为 makespan 改善。

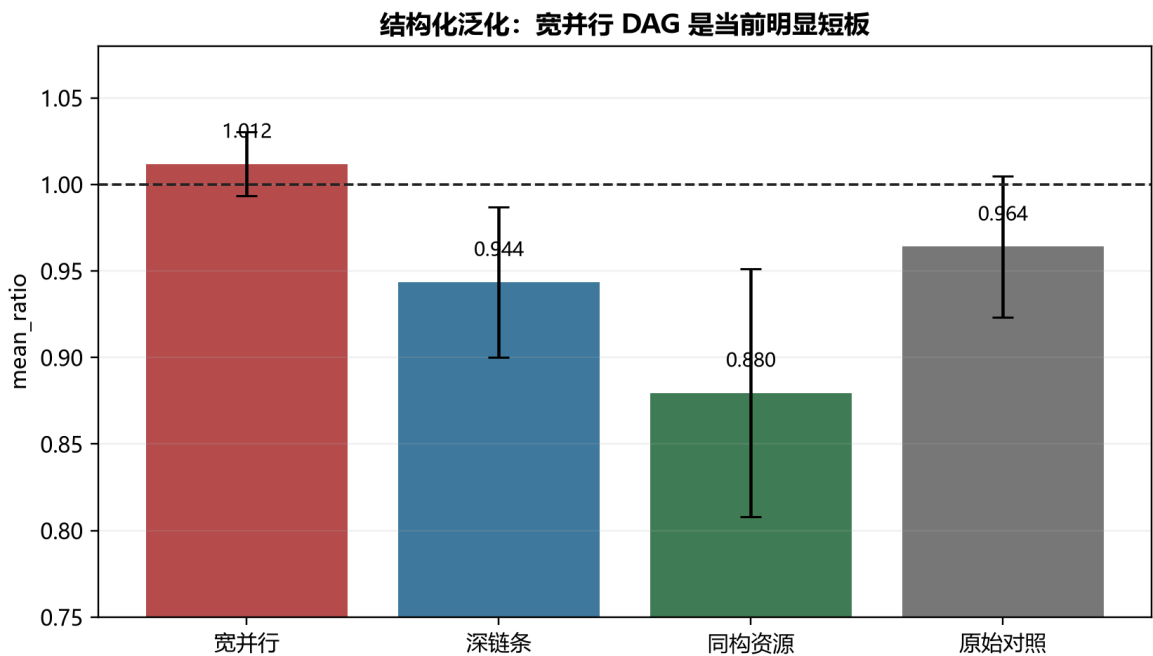


图 5 结构化泛化

6.5 大规模泛化负面结果

为避免把“换模型”和“换规模”混为一谈，项目另训练支持 60 个任务、padding 70 的归一化 MLP。该模型在原规模评测上的 mean_ratio 为 1.336977，在 30 至 60 任务的大规模集上为 1.454469，分别见 `evaluation/results/summary_mlp_normalized_large_small_eval.json` 与 `evaluation/results/summary_mlp_normalized_large.json`。结果没有支持“任务越大越容易反超 HEFT”的假设，表明当前状态表达与训练分布在规模外仍不足。

6.6 openEuler 与测试验证

项目在 `openeuler/openeuler:24.03-lts-sp4` 容器中挂载同一代码与 `checkpoint`，完成依赖安装、`pytest` 和评测。保留证据包括 `evaluation/results/openeuler_validation/environment_info.log`、`evaluation/results/openeuler_validation/pip_freeze.txt`、`evaluation/results/openeuler_validation/pytest_output.log` 与 `evaluation/results/openeuler_validation/evaluate_bestof64_output.log`。其中结构化测试记录为 `19 passed, 17 warnings in 5.33s`，见 `evaluation/results/openEuler_pytest_structural_final.log`。该证据验证的是推理与测试的跨平台可运行性，不声称不同 CPU 上的随机采样结果逐位一致。

第七章 复现、审计与工程交付

7.1 依赖与关键配置

Python 依赖由 requirements.txt 声明：NetworkX、Gymnasium、PyYAML、NumPy、Matplotlib、pytest、SB3-Contrib、Stable-Baselines3、TensorBoard、tqdm、Rich 和 PuLP。最终训练参数、环境范围、checkpoint 与结果路径以 training/configs/ppo_mlp_residual.yaml 为准。

配置/文件	用途
configs/resource_default.yaml	默认云边端资源
env/normalization_stats.json	连续观测字段统计量
training/configs/ppo_mlp_residual.yaml	Residual 训练与常规评测
training/checkpoints/ppo_mlp_residual.zip	最终训练模型
evaluation/scenarios/	固定常规验证集
evaluation/results/autonomous_exploration/	最终搜索与统计证据

7.2 一键运行

Windows PowerShell:

```
[powershell]
powershell -ExecutionPolicy Bypass -File scripts/run_final_pipeline.ps1
```

Linux/openEuler:

```
[bash]
bash scripts/run_final_pipeline.sh
```

现有一键脚本检查依赖与 checkpoint，必要时训练 Residual 模型，生成固定场景并运行 Residual Best-of-64。全新克隆端到端日志 evaluation/results/run_final_pipeline_fresh_clone_full_output.log 记录完成 200704 个 PPO rollout 步、训练耗时 489.71 秒、生成 20 个验证场景并输出最终摘要。该日志属于一次具体机器上的运行证据，不作为普适性能承诺。

7.3 最终 LNS 方案命令

```
[bash]
python evaluation/evaluate_residual_lns.py \
  --config training/configs/ppo_mlp_residual.yaml \
  --model-path training/checkpoints/ppo_mlp_residual \
  --num-samples 64 \
  --lns-iterations 64
```

如需复现 5 次统计，应使用系统熵生成独立种子，逐轮保存 JSON，再运行：

```
[bash]
python evaluation/analyze_residual_lns_direct_repeats.py \
  evaluation/results/autonomous_exploration/direction2_lns/direct_repeat_1.json \
  evaluation/results/autonomous_exploration/direction2_lns/direct_repeat_2.json \
  evaluation/results/autonomous_exploration/direction2_lns/direct_repeat_3.json \
  evaluation/results/autonomous_exploration/direction2_lns/direct_repeat_4.json \
  evaluation/results/autonomous_exploration/direction2_lns/direct_repeat_5.json
```

7.4 测试与报告审计

```
[bash]
python -m pytest tests/ -v
python diagnostics/technical_report_result_audit.py --report docs/技术报告.md
```

审计器检查报告引用路径是否存在、数字是否与 JSON 字段在显示精度内一致、参考文献 URL 是否在核验清单中，并扫描可识别的用户主目录路径。Word 文档由 scripts/generate_report_docx.py 通过 win32com 直接操控 Microsoft Word 生成，随后更新目录、页码和域并导出 PDF 做页面核验。

第八章 局限性、适用边界与未来工作

8.1 已知局限

1. 规模外泛化不足。支持 60 任务的新模型在大规模集上为 1.454469，不能据此声称适用于任意规模；证据见 `evaluation/results/summary_mlp_normalized_large.json`。
2. 宽并行 DAG 较弱。宽并行组为 1.011915，仅 2/5 场景反超 HEFT；证据见 `evaluation/results/structural_generalization/summary_structural_generalization.json`。
3. 推理时间换质量。Best-of-64 与 64 轮 LNS 不是实时毫秒级决策。最终方案适合可接受几十秒离线规划的批调度场景；在线场景需降低样本数或 LNS 预算。
4. best-only 的改进边界。LNS 不会劣化当前解，但也可能停在邻域局部最优；显著性结果只适用于当前场景分布和预算。
5. MILP 覆盖有限。只有 7/10 个小场景证明最优，且 MILP 比较使用较早的 Best-of-16，而非最终 LNS。
6. 训练统计仍可加强。最终 checkpoint 来自单训练种子；推理阶段做了严格重复统计，但未来仍应增加多训练种子和置信区间。

8.2 后续方向

后续工作优先考虑：针对宽并行图的分布增强；可扩展的图指针或集合编码器；自适应 LNS 算子选择；把求解器下界用于训练与停止条件；在真实 openEuler 资源遥测下引入动态负载、故障与迁移开销。任何新方法仍应遵循“先固定评测协议，再看结果”的原则。

第九章 结论

项目的核心贡献不只是得到一个小于 1 的 `mean_ratio`，而是建立了从调度物理一致性、训练诊断、动作空间设计到统计验证的完整工程闭环。最终方案让 HEFT 提供安全排序基准，让 Residual Policy 学习实例相关修正，让 EFT 保证资源放置规则一致，再用 best-only 合法重定位与 LNS 精修高质量初解。

在 5 次系统熵配对评测中，最终方案达到 0.920890 ± 0.001729 ，5 轮均为 20/20 场景反超 HEFT；计算量对齐的 Best-of-128 仍显著更差。与此同时，报告明确保留大规模和宽并行场景的负面结果。由此得到的结论是有限但可验证的：在本项目定义的 8 至 25 任务、默认及若干结构扰动场景上，Residual Best-of-64 加约束搜索精修能够稳定优于 HEFT；该结论不自动外推到任意规模、任意动态负载或实时调度。

参考文献

- [1] Topcuoglu H., Hariri S., Wu M.-Y. Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. IEEE Transactions on Parallel and Distributed Systems, 13(3):260-274, 2002. DOI: <https://doi.org/10.1109/71.993206>.
- [2] Jeon W., Gagrani M., Bartan B., Zeng W. W., Teague H., Zappi P., Lott C. Neural DAG Scheduling via One-Shot Priority Sampling. ICLR 2023. <https://openreview.net/forum?id=WL8FlAugqQ>.
- [3] Ho K.-H., Jheng R.-Y., Wu J.-H., Chiang F., Chen Y.-C., Wu Y.-Y., Wu I.-C. Residual Scheduling: A New Reinforcement Learning Approach to Solving Job Shop Scheduling Problem. arXiv:2309.15517, 2023. <https://arxiv.org/abs/2309.15517>.
- [4] Ropke S., Pisinger D. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. Transportation Science, 40(4):455-472, 2006. DOI: <https://doi.org/10.1287/trsc.1050.0135>.
- [5] Hottung A., Kwon Y.-D., Tierney K. Efficient Active Search for Combinatorial Optimization Problems. ICLR 2022. <https://openreview.net/forum?id=nO5caZwFwYu>.
- [6] Pineau J., Vincent-Lamarre P., Sinha K., Lariviere V., Beygelzimer A., d'Alche-Buc F., Fox E., Larochelle H. Improving Reproducibility in Machine Learning Research: A Report from the NeurIPS 2019 Reproducibility Program. Journal of Machine Learning Research, 22(164):1-20, 2021. <https://www.jmlr.org/papers/v22/20-303.html>.
- [7] 清华大学研究生院. 研究生学位论文写作指南. 2024. <https://www.dhs.tsinghua.edu.cn/wp-content/uploads/2023/12/2024031107044595.pdf>.
- [8] 中国研究生创新实践系列大赛管理平台. 第二届中国研究生操作系统开源创新大赛作品提交说明. <https://cpipc.acge.org.cn/cw/contestNews/detail/2c9080178c7c917b018d1b1a0af61cd6/2c9080189768427f01978ddbf7874abc?page=1>.

附录 A 关键证据索引

结论	原始证据
最 终 LNS 五次统计	evaluation/results/autonomous_exploration/direction2_ins/direct_vs_residual_paired_summary.json
计 算 量 对 齐 Best-of-128	evaluation/results/autonomous_exploration/compute_matched_sampling/paired_comparison_summary.json
自 主 探 索时间线与 预注册式决策	docs/自主探索日志.md
MILP 精确解与界	evaluation/results/milp_optimal_comparison.json
结 构 化 泛化	evaluation/results/structural_generalization/summary_structural_generalization.json
大 规 模 负面结果	evaluation/results/summary_mlp_normalized_large.json
openEuler 环境与测试	evaluation/results/openeuler_validation/
全 新 克 隆一键流水线	evaluation/results/run_final_pipeline_fresh_clone_full_output.log

附录 B 关键代码索引

功能	文件
插入式调度规则	env/scheduling_utils.py
Joint / Ranked / Residual 环境	env/scheduling_env.py env/scheduling_env_ranked.py env/scheduling_env_residual.py
HEFT upward rank	baselines/heft_scheduler.py
Residual logits	policies/residual_ranking_policy.py
Best-of-N	policies/rl_scheduler.py
合法重定位与 LNS	policies/residual_local_search_scheduler.py
MaskablePPO 训练	training/train_ppo.py
最终 LNS 评测	evaluation/evaluate_residual_lns.py
配对统计	evaluation/analyze_residual_lns_direct_repeats.py
报告审计	diagnostics/technical_report_result_audit.py

附录 C 匿名化与数据口径声明

本报告封面不填写学校、指导教师和队员姓名。Word 核心属性中的作者和最后保存者统一写为 **Anonymous**。报告只采用仓库现存原始结果；历史描述与文件冲突时以 JSON/日志为准。单次固定种子结果只用于复现示例，最终结论使用重复评测均值与样本标准差。未证明最优的 MILP 可行解不称为全局最优。所有绝对路径、Windows 用户目录和本机身份信息均不得写入正式报告正文或证据日志。